2026-10-13 MySQL操作数据表操作

- 1. MySQL 表的操作
  - 1.1 创建表
    - 1.1.1 CREATE TABLE 语法
    - 1.1.2 参数说明
    - 1.1.3 创建 users 表
    - 1.1.4 存储引擎与表文件
  - 1.2 查看表结构
    - 1.2.1 DESC 语法
    - 1.2.2 输出字段
    - 1.2.3 查看 users 表
  - 1.3 修改表
    - 1.3.1 修改场景与基本语法
    - 1.3.2 准备示例数据
    - 1.3.3 添加字段
    - 1.3.4 修改字段
    - 1.3.5 删除字段
    - 1.3.6 修改表名与字段名
  - 1.4 删除表
    - 1.4.1 删除表语法
    - 1.4.2 删除表示例
  - 1.5 本章操作总结

1. MySQL 表的操作

数据表是数据库中的对象，表操作通常先选择目标数据库，再进行创建、查看、修改或删除。本章围绕表的生命周期展开，重点是表结构操作（DDL）；其中插入和查询数据仅用于观察修改表结构前后数据的变化。

```
USE test1;
```

1.1 创建表

1.1.1 `CREATE TABLE` 语法

```
CREATE TABLE table_name (
    field1 datatype,
    field2 datatype,
    field3 datatype
) CHARACTER SET 字符集 COLLATE 校验规则 ENGINE 存储引擎;
```

列定义之间用逗号分隔，最后一项后面不加逗号。表级选项放在右括号之后，用于设置表的字符集、校验规则和存储引擎。

1.1.2 参数说明

- `field` 表示列名。
- `datatype` 表示列的数据类型。
- `CHARACTER SET` 指定字符集；如果没有指定，则以所在数据库的字符集为准。
- `COLLATE` 指定校验规则；如果没有指定，则以所在数据库的校验规则为准。
- `ENGINE` 指定存储引擎。不同存储引擎在文件组织、索引和事务等方面可能不同。

1.1.3 创建 `users` 表

```
CREATE TABLE users (
    id INT,
    name VARCHAR(20) COMMENT '用户名',
    password CHAR(32) COMMENT '密码是32位的md5值',
    birthday DATE COMMENT '生日'
) CHARACTER SET utf8 ENGINE MyISAM;
```

这条语句创建了一个包含 `id`、`name`、`password`、`birthday` 四列的表，并使用 `utf8` 字符集和 `MyISAM` 存储引擎。
记录了一个 `age` 字段：

```
age INT COMMENT '年龄'
```

1.1.4 存储引擎与表文件

不同的存储引擎会用不同的方式组织表文件。

存储引擎	表结构	表数据	表索引	组织特点
MyISAM	`users.frm`	`users.MYD`	`users.MYI`	数据和索引分开存储
InnoDB	`users.frm`	`users.ibd`	与数据共同组织	数据和索引放在一起存储

MyISAM 表对应的文件：

```
[root@localhost users]# ll
total 20
-rw-r-----. 1 mysql mysql   61 Oct 14 03:13 db.opt
-rw-r-----. 1 mysql mysql 8727 Oct 14 04:06 users.frm
-rw-r-----. 1 mysql mysql    0 Oct 14 04:06 users.MYD
-rw-r-----. 1 mysql mysql 1024 Oct 14 04:06 users.MYI
```

InnoDB 表对应的文件：

```
[root@localhost mysql]# cd text1/
[root@localhost text1]# ll
total 112
-rw-r-----. 1 mysql mysql    61 Oct 14 04:12 db.opt
-rw-r-----. 1 mysql mysql  8727 Oct 14 04:14 users.frm
-rw-r-----. 1 mysql mysql 98304 Oct 14 04:14 users.ibd
[root@localhost text1]#
```

因此，创建表时选择不同存储引擎，磁盘上的文件组成也会不同。其中最直接的差异是：MyISAM 将数据和索引分别放在 `.MYD` 与 `.MYI` 文件中，而 InnoDB 将数据和索引共同存放在 `.ibd` 表空间中。这一差异也为后续理解索引和事务打下基础。

> 在较新的 MySQL 版本中，表定义通常由数据字典管理，不再依赖 `.frm` 文件，但 `.ibd` 等表空间文件及两种引擎的组织差异仍可用来理解课程内容。

1.2 查看表结构

1.2.1 `DESC` 语法

```
DESC 表名;
```

`DESC` 用于查看表中包含哪些字段，以及字段类型、空值属性、索引信息和默认值等。

1.2.2 输出字段

输出列	含义
`Field`	字段名字
`Type`	字段类型
`Null`	是否允许为空
`Key`	索引类型
`Default`	默认值
`Extra`	扩充信息

1.2.3 查看 `users` 表

```
DESC users;
```

预期结果：

Field	Type	Null	Key	Default	Extra
`id`	`int(11)`	YES		NULL	
`name`	`varchar(20)`	YES		NULL	
`password`	`char(32)`	YES		NULL	
`birthday`	`date`	YES		NULL	

原笔记中带 `age` 字段的另一版查看结果：

```
mysql> desc users;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| id       | int(11)     | YES  |     | NULL    |       |
| name     | varchar(20) | YES  |     | NULL    |       |
| password | char(32)    | YES  |     | NULL    |       |
| birthday | date        | YES  |     | NULL    |       |
| age      | int(11)     | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
5 rows in set (0.19 sec)
```

> 输出中的类型显示可能随 MySQL 版本变化，例如整数类型的显示宽度在不同版本中可能不同；字段含义和表结构本身不受这一显示差异影响。

1.3 修改表

1.3.1 修改场景与基本语法

在实际开发中，经常需要修改表结构，例如修改字段名称、字段大小、字段类型、表的字符集或存储引擎，也可能需要添加字段、删除字段。

基本语法：

```
ALTER TABLE tablename ADD (
    column datatype [DEFAULT expr][, column datatype]...
);

ALTER TABLE tablename MODIFY (
    column datatype [DEFAULT expr][, column datatype]...
);

ALTER TABLE tablename DROP (column);
```

三类操作分别对应添加字段、修改字段定义和删除字段。实际使用时，应根据操作写出完整的列定义。

1.3.2 准备示例数据

为了观察添加字段后原数据是否保留，先向 `users` 表中插入两条记录：

```
INSERT INTO users
VALUES
    (1, 'a', 'b', '1982-01-04'),
    (2, 'b', 'c', '1984-01-04');
```

这里的值按建表时的列顺序依次对应 `id`、`name`、`password` 和 `birthday`。

1.3.3 添加字段

添加一个用于保存图片路径的字段，并把它放在 `birthday` 后面：

```
ALTER TABLE users
ADD assets VARCHAR(100) COMMENT '图片路径' AFTER birthday;
```

查看修改后的表结构：

```
DESC users;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	YES		NULL	
name	varchar(20)	YES		NULL	
password	char(32)	YES		NULL	
birthday	date	YES		NULL	
assets	varchar(100)	YES		NULL	

`AFTER birthday` 只决定新字段在表中的位置；不加 `AFTER` 时，新字段通常追加到末尾。添加字段不会删除原有记录：

```
SELECT * FROM users;
```

id	name	password	birthday	assets
1	a	b	1982-01-04	NULL
2	b	c	1984-01-04	NULL

原来的两行数据仍然存在，新添加的 `assets` 列暂时为 `NULL`。

1.3.4 修改字段

把 `name` 字段的长度从 20 改成 60：

```
ALTER TABLE users MODIFY name VARCHAR(60);
```

再次查看表结构：

```
DESC users;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	YES		NULL	
name	varchar(60)	YES		NULL	
password	char(32)	YES		NULL	
birthday	date	YES		NULL	
assets	varchar(100)	YES		NULL	

`name` 的显示类型已经变成 `varchar(60)`。`MODIFY` 会用新定义覆盖原字段定义，因此修改时应把希望保留的类型、默认值、注释和约束完整写出。

1.3.5 删除字段

删除 `password` 列：

```
ALTER TABLE users DROP password;
```

查看结果：

```
DESC users;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	YES		NULL	
name	varchar(60)	YES		NULL	
birthday	date	YES		NULL	
assets	varchar(100)	YES		NULL	

> 删除字段一定要小心。删除字段时，该列中的全部数据也会一起删除，不能再通过普通查询找回。

1.3.6 修改表名与字段名

把表名从 `users` 改成 `employee`：

```
ALTER TABLE users RENAME TO employee;
```

语法中的 `TO` 可以省略。改名后仍可查询原来的记录：

```
SELECT * FROM employee;
```

id	name	birthday	assets
1	a	1982-01-04	NULL
2	b	1984-01-04	NULL

接着把 `name` 列改名为 `xingming`：

```
ALTER TABLE employee CHANGE name xingming VARCHAR(60);
```

查看修改后的结构：

```
DESC employee;
```

Field	Type	Null	Key	Default	Extra
`id`	`int(11)`	YES		NULL	
`xingming`	`varchar(60)`	YES		NULL	
`birthday`	`date`	YES		NULL	
`assets`	`varchar(100)`	YES		NULL	

`CHANGE` 同时完成字段改名和字段重定义，因此必须写出旧字段名、新字段名以及完整的新定义。

操作	语法	是否改名	字段定义的处理
修改已有字段	`MODIFY`	否	用新定义覆盖原定义
修改字段名	`CHANGE`	是	写旧字段名、新字段名和完整定义

1.4 删除表

1.4.1 删除表语法

```
DROP [TEMPORARY] TABLE [IF EXISTS] tbl_name [, tbl_name] ...;
```

- `TEMPORARY` 用于删除临时表。
- `IF EXISTS` 可以避免表不存在时因直接删除而产生错误。
- `tbl_name [, tbl_name] ...` 表示一次可以指定一个或多个表。

1.4.2 删除表示例

```
DROP TABLE t1;
```

`DROP TABLE` 删除的是整张表，不只是表中的某一行或某一列。表结构、数据以及相关索引都会随表一起被删除，因此执行前应确认目标表并做好必要备份。

1.5 本章操作总结

本章通过 `users` 表的完整操作过程，串联了表结构查看与变更：

```
创建 users 表
    -> DESC 查看结构
    -> 添加 assets 字段
    -> 修改 name 字段长度
    -> 删除 password 字段
    -> 修改表名为 employee
    -> 修改 name 字段名为 xingming
    -> DROP TABLE 删除整表
```

目的	常用语句	作用对象
创建表	`CREATE TABLE`	新表及列定义
查看表结构	`DESC 表名;`	表的字段信息
添加字段	`ALTER TABLE ... ADD ...`	表中的某一列
修改字段定义	`ALTER TABLE ... MODIFY ...`	指定列的完整定义
重命名字段	`ALTER TABLE ... CHANGE ...`	旧字段名、新字段名及定义
删除字段	`ALTER TABLE ... DROP ...`	表中的某一列及其数据
重命名表	`ALTER TABLE ... RENAME TO ...`	整张表的名称
删除表	`DROP TABLE`	整张表的结构、数据与索引

需要始终区分：`ALTER TABLE ... DROP` 删除的是字段，`DROP TABLE` 删除的是整张表；`INSERT`、`SELECT` 等语句操作表中的记录，而本章的 `CREATE`、`ALTER`、`DROP` 主要操作表结构。